

W1 BIG DATA MOTIVATION
Data science problem: combines data + method + decision, not just computing summary stats. Need stakeholder goal, data provenance, measurable success, limits.
Big data 5V: Volume size, **Velocity** arrival speed, **Veracity** relational/JSON/text/image/geo/time, **Veracity** uncertainty/quality, **Value** decision utility.
Design thinking: empathise → define problem → ideate → prototype → test. Good for authentic DS because it links analysis to user needs and iteratively refines assumptions.
Past-paper wording: "mean/std of marks" alone is descriptive statistics; "identify at-risk students using learning history" is a DS problem because data supports prediction/action.
Data types: nominal unordered labels, use count/mode/bar; **ordinal** ordered labels, median/IQR ok, intervals not guaranteed; **interval/ratio** numeric, mean/SD meaningful; **id** not numeric measure.
Quality dimensions: completeness, validity, consistency, uniqueness, timeliness, accuracy. Detection: missing markers ?, NA, !; domain rules; duplicate keys; impossible dates; unit mismatch; referential-integrity breaks.
Pitfalls: bad data, data before theory, correlation ≠ causation, regression to mean, harking groups, overclaiming precision. Explain with consequence + mitigation. EDA cycle: ingest → inspect schema/types → profile nulls/ranges/outliers → clean/standardise → summarise/visualise → document assumptions.

Type	Stats	Plot
------	-------	------

nominal/count/mode bar
ordinal median/rank bar
numeric/mean/SD hist/box
time trend/season line
density/dist map

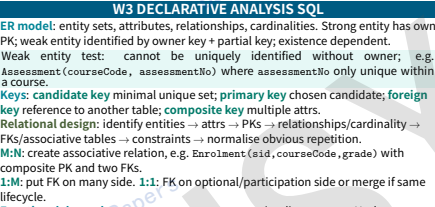
Story styles: change-over time, drill-down, zoom-out, contrast, intersection, choices, outliers. Choose by stakeholder question; policy history often needs time trend + population zoom-out.
Exam answer shape: state assumption, define variable/data method, choose method, justify, name risk/limitation.

W2 RELATIONAL DATA & SQL
Relation: table with attributes/columns and tuples/rows; schema fixes names/types/constraints; instance is current rows. **Entity integrity:** PK not null/unique. **Referential integrity:** FK values match referenced PK or allowed null. **SQL declarative:** query states what result, optimizer chooses how: scan vs index, join order, hash/sort join.
SELECT core: SELECT attr FROM tables WHERE row-filter GROUP BY keys ORDER BY sort; **Joins:** FROM tables logical order: FROM/JOIN → WHERE → GROUP → HAVING → SELECT → ORDER.
WHERE filters rows before grouping; **HAVING filters groups** after aggregation. **NULL** uses IS NULL, not = NULL.
DDL/DML: DDL defines structure: CREATE, ALTER, DROP; DML manipulates/queries data: SELECT, INSERT, UPDATE, DELETE.
Join: B JOIN S ON B.f= S.f; ON f= S.f; inner drops non-matches; left keeps left rows w/ nulls. Cartesian product if missing join condition.
Aggregation: COUNT(*) rows; COUNT(col) non-null values; COUNT(DISTINCT id) de-duplicates. Group by: non-aggregated selected attr.
Retrieval: row-level predicate chooses tuples; column-level projection chooses attr. **SELECT** * returns all cols, prevents covering-only plans and can expose unnecessary data.
Common query: SELECT name, restaurant, rating FROM Rating WHERE rating>15 ORDER BY name ASC;
Set/bag: SQL is bag/multiset by default; duplicates remain unless DISTINCT. UNION removes duplicates; UNION ALL keeps.

Need	SQL pattern
top 5 missing per group ORDER BY x DESC LIMIT 5	ORDER BY x DESC LIMIT 5
per group ORDER BY x	ORDER BY x DESC LIMIT 5
post agg HAVING COUNT(*)>1	HAVING COUNT(*)>1

Trap: string literal quotes use 'rating', but column name uses rating; WHERE 'RATING'>15 compares a literal, not column.
W3 DECLARATIVE ANALYSIS SQL
ER model: entity sets, attributes, relationships, cardinalities. Strong entity has own PK; weak entity identified by owner key + partial key; existence dependent.
Weak entity test: cannot be uniquely identified without owner; e.g. Assessment(CourseCode, assessmentNo) where assessmentNo only unique within a course.
Keys: candidate key minimal unique set; **primary** key chosen candidate; **foreign** key reference to another table; **composite** key multiple attr.
Relational design: identify entities → attr → PKs → relationships/cardinality → FKs/associative tables → constraints → normalise obvious repetition.
M:N: create associative relation, e.g. Enrolment(said, courseCode, grade) with composite PK and two FKs.
1:M: put FK on many side. **1:1:** FK on optional/participation side or merge if same lifecycle.

Functional dependency: X → Y means same X implies same Y. Update anomalies when facts repeated.
Normalisation idea: separate entities to reduce redundancy while preserving ability to join. Do not store derived totals unless justified/cached.



ERD answer: draw entities, underline PK, mark FK/partial key, show cardinality/participation, then translate into table schemas.
Common mistake: FK is not "the same as PK"; it is an attribute constrained to match referenced key values, allowing relationships across tables.
Integrity examples: orphan assessments = FK violation; duplicate said = PK violation; missing required key = entity-integrity violation.
Exam strategy: for schema fill-in, first mark independent entities, then weak/associative entities; never invent surrogate key if question tests partial/composite key.

W4 PYTHON CLEAN/EXPLORE
Pandas pipeline: read_csv → head/info/describe → parse types → normalise labels → handle missing/duplicates/outliers → group/merge/plot.
Type conversion: CSV reader returns strings; convert because numeric comparisons, dates, sorting, joins, and aggregation depend on correct semantics. '100' < '20' lexicographically.
pd.to_numeric(x, errors='coerce') turns invalid numeric tokens into NaN; pd.to_datetime(...) enables date arithmetic/extract month.

Missing data: MCAR/MAR/MNAR. Actions: standardise markers, keep missing information, impute cautiously, drop only if justified. Distinguishes real zero from missing placeholder.
Duplicates: exact duplicate rows vs duplicate entity IDs with conflicting attrs. Use drop_duplicates() only after defining entity identity.
Groupby: df.groupby('sensor')['value'].mean() → SELECT sensor, AVG(value) FROM T GROUP BY sensor;
Merge: inner/left/right/outer. Check key uniqueness and row counts before/after. Many-to-many join can explode rows.
Plot workflow: create derived column → aggregate counts/stat → choose plot → label axes/title → interpret pattern, not just display.

Issue	Detect	Fix
r value counts	coerce/NA	
unit mix	rules/rules	convert
key count	dedupe	dedupe
box rule	verify	verify
date str	dtype	parse

Jupyter practice: reproducible notebooks need ordered cells, fixed random seeds, visible assumptions, no hidden manual edits.
Quality answer: measure dimension + method: incomplete → null ratio; inconsistent → cross-field rule violations; duplicate → repeated keys; out-of-date → timestamp age.

W5 INDEX & PARTITION
Index: auxiliary access path mapping key values to rows/pages. Speeds reads for selective predicates, slows writes/storage.
Hash index: equality lookup x=...; poor for range/order. **B+ tree:** equality, range >, BETWEEN, prefix order, min/max, sorted scan.
For WHERE day=D AND start BETWEEN A AND C, use composite B+ tree (day, start, on); equality column first, then range column.
Clustered index: physical row order follows index; good for range scans/locality; only one clustered order; inserts/updates may cost more. **Unclustered:** secondary pointers; many possible; may require random fetch.
Covering index: includes all columns needed by query, enabling index-only access. For SELECT name, restaurant, rating WHERE rating>15 ORDER BY name, one candidate: B+ tree on (rating, name, restaurant) OR (rating).
INCLUDE (name, restaurant): Improves queries aligned with clustered order; may hurt unrelated predicates, updates, and competing access patterns.
Index design: list workload → filter/order/group cols → equality before range → include projected cols if covering → consider selectivity/update cost → test plan.
Partitioning: split table/data by range, hash, list, composite. Goal: pruning, parallelism, locality, manageability.
Horizontal partition: rows split. **Vertical partition:** columns split. **Sharding:** distributed horizontal partition with routing.
MapReduce: map emits key-value pairs; shuffle groups by key; reduce aggregates. Good for embarrassingly parallel batch jobs; overhead bad for tiny/interactive tasks.

Predicate	Index	Why
id=7	hash/B+	equality
x>10	B+	range
ORDER BY t	B+	sorted
spatial	GiST/R-tree	boxes
text	inverted	tokens

Exam pitfall: low-selectivity predicates may not benefit; optimizer may choose table scan.
W6 WEB SCRAPING
HTML DOM: tree of elements, tags, attrs, text. CSS selectors locate nodes: div.card, #id, a[href].
BeautifulSoup: soup=BeautifulSoup(html, 'html.parser'); find/find_all/select; get_text(strip=True); attr via tag['href'] or tag.get('href').
Robust pattern: check node exists before chaining: topics = [s.text for s in div.find_all('span')] if div else []
Static vs dynamic: static HTML fetched by requests; JS-rendered content may require browser/API endpoint. Prefer official API when available.
Ethics/legal: obey ToS/robots where applicable, throttle requests, identify user-agent, avoid personal/sensitive data, cache results, cite source.
Scrape-to-DF: request → parse cards → extract fields/list attrs → normalise missing URLs/text → create DataFrame → parse dates → aggregate/plot.
URL handling: resolve relative links with urljoin; beware pagination, duplicates, encoding, redirects, and rate limits.
Parsing errors: AttributeError when calling method on None; KeyError for missing attr; malformed HTML may still parse into unexpected tree.

Need	Code idea
text	tag.get_text(strip=True)
link	a.get('href')
many	select('item')
optional if tag else None	optional if tag else None
table	read_html()

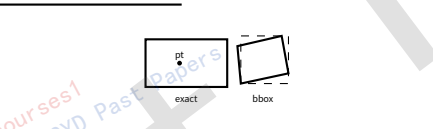
Past-paper tip: explain each plotting step: parse month, group/count, bar plot, labels/title; missing topic: div needs guard or empty list.
Data risk: web pages change; scraper must be maintainable and tested against small samples.

W7 API & NOSQL
API: contract for programmatic access; request endpoint + method + params/headers/auth; response status + body, often JSON.
REST flow: read docs → build URL/params → handle auth/rate limits → check status code → parse JSON → pagination → normalise nested records.
JSON: key-value, arrays, nested objects, schema-flexible. **XML:** user-defined tags/attrs, hierarchical. Both semi-structured/self-describing; not fixed relational schema.
Semi-structured data can carry structure with data, often nested/repeated; it still needs validation before analysis.
NoSQL families: key-value, document, wide-column, graph. Choose by access pattern: document for nested entities, graph for relationships, KV for lookup/cache.
Schema-on-read: interpret structure when reading; flexible but shifts validation burden to application/query. **Schema-on-write:** relational constraints enforced before storage.
Flatten JSON: json_normalize, explode arrays, preserve IDs, handle missing keys, document assumptions. Nested arrays can create one-to-many rows.
API errors: 400 bad request, 401 auth, 403 forbidden, 404 missing, 429 rate limit, 5xx server. Retry with backoff only for transient failures.
Data integration: map source IDs, harmonise types/units/timezones, deduplicate, reconcile conflicting source values, maintain lineage.

Model	Strength	Risk
relational	joins/constraints	rigid
document	nested/flexible	dup schema
graph	paths	scale/query
KV	fast lookup	few queries

Answer note: when asked JSON vs XML, mention both are semi-structured; XML uses tags/elements; JSON uses key-value/object/array syntax; both may use user-defined names.
W8 GEOSPATIAL DATA
Geometry: point/line/polygon; attributes stored with geometry. CRS/SRID defines coordinates and distance meaning. Latitude/longitude degrees are not metres. Before spatial join, transform to common CRS: e.g. ST_Transform(S.location, 4283) if regions use SRID 4283.
Spatial predicates: ST_Contains(poly, pt), ST_Within(pt, poly), ST_Intersects, ST_Distance, ST_DWithin. Boundary semantics may differ.
Spatial join: candidate geometries via bbox index → exact predicate refine → aggregate/count/distinct.
Spatial index: R-tree/GiST stores bounding boxes to prune comparisons; exact geometry still checked after candidate filtering.
Distinct schools by region: SELECT r.region_name, COUNT(DISTINCT s.school_id) AS s FROM Regions r JOIN Schools s ON ST_Contains(r.boundary, s.location) WHERE r.state='NSW' GROUP BY r.region_name ORDER BY s DESC LIMIT 5;
COUNT(DISTINCT) safer than COUNT(*) when raw import duplicates same school.
Common issues: swapped lat/lon, invalid polygons, multipolygons, points on boundary, missing CRS, projected vs geographic units, duplicate place names.

Task	Method
region membership	point-in-polygon
nearest facility	distance/KNN
hotspot	grid/kernel density
route corridor	buffer/index



Exam phrase: spatial index helps filter candidates by bounding boxes; refine stage evaluates true geometry predicate.

W9 TEXT PROCESSING
Pipeline: collect docs → tokenize → normalise → remove stop/punct if suitable → stem/lemmatise → vectorise → model/evaluate.
Tokenisation: split text into units; choices affect hashtags, URLs, contractions, punctuation, Chinese segmentation. **Normalisation:** lower-case, lemmatise/stem, spelling, encoding.
Bag-of-Words: sparse counts/indicator, interpretable, ignores order/context. **n-grams:** preserve short local phrases but higher dimensionality.
 $f(t_i, d)$ measures term frequency: $idf(t_i) = \log(N/df_i)$; $tfidf = tf \times idf$.
Rare-across-docs terms get larger IDF; terms in many docs get lower IDF.
IDF change: if term appears in more documents relative to N, IDF decreases; if df same while N increases, IDF increases.
Count/Vectorizer: vocabulary maps token to feature index; output matrix rows=docs, cols=tokens. binary=True records presence not count.
Embeddings: word2vec dense word vectors, semantic similarity but context-insensitive; BERT contextual sentence/token embeddings, powerful but heavier/less interpretable.

Method	Strength	Limit
BoW	simple/explain	no order
TF-IDF	downweights common	surface match
word2vec	semantic word	single sense
BERT	context/sentence	cos/coscity

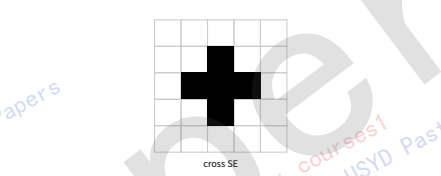
Past-paper choice: for scoring clue sentences/paraphrases, BERT usually best; word2vec acceptable simpler; BoW weakest when synonyms/context matter.
Text pitfalls: leakage via labels in text, over-clearing meaningful tokens, domain stopwords, imbalanced classes, vocabulary shift, encoding issues.
Similarity: cosine $\cos(x, y) = x \cdot y / (||x|| ||y||)$ common for sparse/dense vectors; magnitude removed, direction compared.
Answer style: state representation, what information it preserves, what it loses, and why that matters for the task.

W10 TIME SERIES/HEALTH
Time series: ordered observations; dependence means random split leaks future. Use chronological split/rolling origin.
Components: trend, seasonality, cycle, noise; health data often has reporting delays, censoring, missingness, regime changes, privacy constraints.
Datetime features: parse timestamps, sort, set index, resample, aggregate, rolling mean, lag variables, handle timezone/irregular sampling.
Never compute imputation/scaling/features using future test values; leakage is especially easy with rolling/lag features.
Resampling: downsample aggregates (hour → day sum/min); upsample creates gaps requiring imputation/forward fill depending on meaning.
Rolling window: moving mean smooths short-term noise but lags change; window size controls smoothness/responsiveness.
Health data: repeated measures per patient, missing not at random, measurement error, class imbalance, protected attributes, de-identification, consent/ethics.
EDA questions: is sampling regular? gaps/outliers? changes in collection method? seasonality? autocorrelation? subgroup differences?

Issue	Risk	Mitigation
delay	biased recent	nowcast/flag
MNAR	wrong impute	sensitivity
privacy re-id	shift	aggregate/suppress
model decay		monitor

Forecast baseline: naive last value, seasonal naive, moving average. Advanced model must beat baseline and give uncertainty.
Exam answer: for health/time tasks, mention ethical use, privacy, bias, clinically meaningful metrics, and temporal validation.
W11 IMAGE PROCESSING
Image: pixel grid; grayscale intensity; RGB channels; binary image has two classes often 0/1 foreground/background.
Histogram threshold: with bimodal background/object peaks, choose global threshold in valley separating peaks. Otsu chooses threshold maximising between-class variance.
Binary threshold: $g(x, y) = 1$ if $f(x, y) \geq T$, else 0. Choice of T controls false foreground/background.

Morphology: structuring element probes binary image. **Dilation** expands foreground/fills small gaps; **Erosion** shrinks foreground/removes small objects. Opening: erosion then dilation; removes small bright noise while roughly restoring larger object. Closing: dilation then erosion; fills small holes/gaps.
Past-paper correction: dilation does not remove salt/bright noise; it grows bright pixels. To remove small bright isolated noise use opening.



Feature extraction: edges, corners, texture, colour histograms, connected components, shape descriptors, CNN embeddings. Match feature to task and invariances.
Image pitfalls: illumination variation, blur/noise, threshold not transferable, class imbalance, annotation error, compression artifacts.
Operation Effect Use
threshold gray → binary segment
dilate grow/ls bridge/fill
erode shrink/ls remove small
open erode-dilate denoise
close dilate+erode fill holes

Explain binary: each pixel assigned to exactly one of two classes; display colour black/white is visual encoding, not necessarily original intensity.

W12 PRIVACY & CHALLENGES
Privacy: protect individuals/groups from re-identification, misuse, unauthorised disclosure. Big data increases linkage risk across sources.
Anonymisation: remove direct identifiers; pseudonymisation replaces IDs; aggregation/generalisation/suppression reduce detail; k-anonymity requires each quasi-identifier group size ≥ k.
De-identified k: risk-free. Quasi-identifiers (postcode, DOB, gender, rare condition) can re-identify when linked with external data.
Challenges: scale, distributed storage/compute, heterogeneity, data quality, privacy, bias/fairness, reproducibility, energy/cost, governance.
Big-data architecture: ingest → store raw → clean/validate → partition/index → distributed compute → model/aggregate → monitor/audit.
Compression: reduces storage/I/O; may cost CPU; lossless required for exact analytics, lossy may be ok for images/signals with caveats.
Distributed processing: partition data and move compute to data; shuffle is expensive; skewed keys create stragglers.
Security controls: access control, encryption in transit/at rest, audit logs, least privilege, data retention limits.
Fairness: biased data collection and proxy variables can create discriminatory decisions; evaluate by subgroup and document harms.

Method	Protects	Cost
suppress	rare values	utility loss
generalise	coarse stats	coarser stats
aggregate	individuals	less granular
noise	membership	accuracy
access ctrl	misuse	admin

Final-exam checklist: define terms, write SQL precisely, justify index/CRS/representation, mention quality/privacy, compare methods with task-specific tradeoff, give robust edge-case handling.
One-line templates: "Use B+ tree because page predicate"; "Transform CRS before spatial predicate"; "BERT preserves context but is heavier"; "Opening removes isolated foreground noise"; "COUNT DISTINCT avoids duplicate-source inflation".
Mini example SQL: if raw table has repeated school rows, group by region after spatial join and use COUNT(DISTINCT school_id). If question asks why, answer: duplicates inflate COUNT(*); distinct matches semantic unit.
Mini example index: query WHERE rating>15 ORDER BY name SELECT name, rating. B+ tree supports range; covering index avoids base-row fetch; SELECT * usually prevents covering because all attrs required.
Mini example TF-IDF: $N = 3$, $df(q) = 2$ → $idf = \log(3/2)$; add doc containing q, $N = 4$, $df = 3$ → $idf = \log(4/3)$ lower; add doc not containing rare term, df same, IDF higher.
Mini example image: for salt noise in foreground pixels, opening = erosion removes isolated 1, dilation restores main object. Closing would instead fill holes/gaps and may keep/grow noise.
Mini example API: for paginated JSON, loop while next exists; append records; normalise nested arrays into child table; keep source URL/timestamp for lineage.
Explain vs compute: definitions earn little unless tied to given data. Use table row names, attribute names, CRS values, threshold valley, or token counts from prompt.
Assumptions: state when boundary inclusion, NULL semantics, CRS, duplicate identity, or threshold rule is ambiguous. Reasonable assumptions preserve marks.
Communication: report method + result + limitation. Example: "The map is exploratory; hotspots may reflect reporting bias and population exposure, not true risk alone."
SQL safety: qualify attrs after joins; avoid implicit joins; avoid filtering after aggregation unless HAVING; avoid comparing numbers stored as strings.
Data ethics: minimum necessary data, purpose limitation, consent/legal basis, secure storage, retention/deletion, subgroup fairness check, documentation.
Fast final revision: W5 SQL syntax; W3 PK/FK/weak ERD; W5 B+vs hash/cluster/cover; W8 CRS + spatial index; W9 TF-IDF/embeddings; W11 threshold/morphology; W12 privacy.

Issue	Tradeoff	Assumption
justify	compare	strength/limit/task
design	schema/index/workflow	
metric/rule/query		
detect	missing/error guard	
robust	privacy	Q/lineage/control

SQL queries: SELECT k, COUNT(*) FROM T WHERE cond GROUP BY k HAVING COUNT(*)>n ORDER BY COUNT(*) DESC; SELECT ... FROM A JOIN B ON A.id=B.id.
Cleaning skeletons: define boundary inclusion, NULL semantics, CRS, duplicate names/duplicates/outliers → apply domain rules → log rows changed → re-profile.
Index skeleton: equality cols first, one range col next, projected cols as include/covering, avoid over-indexing volatile tables.

Spatial skeleton: verify CRS → transform → spatial index candidate bbox → exact predicate → aggregate distinct entities.
Text skeleton: tokenise/normalise → vectorise → compare BoW/TF-IDF/embedding → evaluate on held-out data; mention vocabulary/context units.
Image skeleton: inspect hist → threshold → opening/closing → erosion/dilation → connected components/features → threshold → binary.
Common MQC cues: ordinal, semi-structured = nested/self-describing, two pixel classes.
Long-answer cue: if asked "explain" and one limitation/counterexample. When stuck: identify data model, answers are method choice + why + etc.

USYD各科往届题库



lost DATA2001 an-